มินิโปรเจกต์ Image Processing
ระบบตรวจความหนาแน่นจราจร
จากกล้อง CCTV เทศบาลนครรังสิต

สรุปแนวคิด สถาปัตยกรรม การเชื่อมโยงกับเนื้อหาสัปดาห์ 1–9 และการแบ่งงานสมาชิก 5 คน · 21 ตุลาคม 2569

1. แนวคิดโดยรวม

ดึงภาพนิ่ง (snapshot) จากกล้อง CCTV ทุก 5–15 นาที แล้วใช้ ROI mask ตัดอาคารและส่วนที่ไม่เกี่ยวข้องออก เหลือไว้เฉพาะพื้นที่ถนนของแต่ละเลน (ขาเข้า/ขาออก) จากนั้นแบ่งส่วน (segment) พิกเซลที่เป็น **รถ** ออกจากพิกเซลที่เป็น **ถนน** และคำนวณสัดส่วนพื้นที่ที่ถูกรถครอบครอง (occupancy) ถ้าถนนเหลือมากแปลว่ารถโล่ง ถ้าพื้นที่เต็มไปด้วยรถแปลว่ารถติด ให้รันระบบต่อเนื่องอย่างน้อย 1 วัน เพื่อสรุปว่าช่วงเวลาไหนรถติดและช่วงไหนโล่ง



ภาพตัวอย่างจากกล้อง (21/10/2569 11:01 น.)

- ภาพ JPEG ขนาด 720×576 ประมาณ 120 KB ต่อภาพ
- มุมซ้ายบนมีเวลาประทับอยู่ในภาพ ต้องตัดออกก่อนประมวลผล
- ถนนแบ่งเป็นฝั่งซ้ายและขวา คั่นด้วยเกาะกลางที่มีต้นไม้
- ช่วงล่างมีซุ้มประตูเมืองบังถนน จึงควรเลือก ROI ช่วงเหนือซุ้ม
- รถที่อยู่ไกลกล้องดูเล็กกว่ารถที่อยู่ใกล้ (perspective)

แหล่งข้อมูล: ดึงภาพได้โดยตรงจากลิงก์ snapshot (ทดสอบแล้วว่าใช้งานได้)

```
https://ipcamlive.com/player/snapshot.php?alias=6695edb811079
```

เรื่องเว็บฮุก: เว็บนี้ไม่ได้ส่งข้อมูลมาหาเราเอง เราต้องเป็นฝ่ายไปดึงภาพเองตามรอบเวลา (polling) ซึ่งทำง่ายกว่าเว็บฮุก ถ้าดึงทุก 5 นาทีจะได้ 288 ภาพต่อวัน รวมประมาณ 35 MB

2. สถาปัตยกรรมระบบ

1	**Collector**	ดึง snapshot ทุก 5 นาทีด้วย `requests` แล้วบันทึกเป็น `data/raw/2026-10-21/1105.jpg`
2	**Preprocess**	ตัดส่วนเวลาที่ประทับ → แปลงเป็น grayscale → median filter → ปรับ histogram
3	**ROI Mask**	ภาพขาวดำของแต่ละเลน (ขาเข้า/ขาออก) เพื่อปิดอาคาร ซุ้มประตู และท้องฟ้า
4	**Segmentation**	หา absdiff กับภาพถนนว่าง → Otsu threshold (ใช้ edge density จาก Sobel, ค่า S ใน HSV หรือ K-means เสริมได้)
5	**Morphology**	opening ลบจุดรบกวน → closing อุดรูในตัวรถ

6	**Metric**	occupancy = พิกเซลรถ ÷ พิกเซลถนนใน ROI → แบ่งเป็น โล่ง / ปานกลาง / ติด
7	**บันทึก CSV**	`timestamp, lane, occupancy, level, brightness`
8	**วิเคราะห์ผล**	pandas + matplotlib → กราฟ 24 ชม., heatmap ตามช่วงเวลา, ภาพ overlay

ไลบรารีที่ใช้: `opencv-python` , `numpy` , `requests` , `pandas` , `matplotlib` ห้าตัวนี้พอ

รายละเอียดสำคัญแต่ละขั้น

- **Collector:** ใช้ลูป `while True` กับ `time.sleep(300)` หรือตั้งใน Windows Task Scheduler ก็ได้ ใส่ try/except เผื่อเน็ตหลุดหรือกล้องล่ม และตั้งชื่อไฟล์ตามเวลา **ควรเริ่มเก็บข้อมูลตั้งแต่วันแรก** ถ้าทำได้ให้เก็บทั้งวันธรรมดาและวันหยุด
- **ภาพถนนว่าง (background) คือหัวใจของงาน:** เอาภาพหลายสิบภาพมาหาค่า median ทีละพิกเซลตามแกนเวลา ด้วย `np.median(stack, axis=0)` เนื่องจากรถอยู่ตำแหน่งเดิมไม่นาน ค่าที่ได้จึงเป็นภาพถนนเปล่า นี่คือแนวคิด median filter จากสัปดาห์ 3 แต่ทำตามเวลาแทนตามพื้นที่ ควรทำ background แยกกลางวันกับกลางคืน
- **ROI:** ใช้ `cv2.fillPoly` วาดรูปหลายเหลี่ยมครอบแต่ละเลนครั้งเดียว แล้วบันทึกเป็นไฟล์ mask
- **ปัญหา perspective:** ใช้ `cv2.warpPerspective` ดึงเลนให้เป็นสี่เหลี่ยมมุมมองจากด้านบน (bird's-eye) หรือแบ่ง ROI เป็นช่วงใกล้/กลาง/ไกล แล้วถ่วงน้ำหนักแต่ละช่วง
- **เกณฑ์แบ่งระดับ:** เริ่มจากค่าประมาณ เช่น ต่ำกว่า 20% = โล่ง, 20–45% = ปานกลาง, มากกว่า 45% = ติด จากนั้นปรับเกณฑ์จากภาพจริงที่คนติดป้ายระดับไว้ 50–100 ภาพ แล้วประเมินผลด้วย confusion matrix

ไอเดียเสริม (ทำง่ายแต่ได้ผลดี): ทุกรอบให้ถ่าย 2 ภาพห่างกัน 15 วินาที แล้วหา absdiff ระหว่างสองภาพ
• ถ้ารถหนาแน่นแต่ภาพแทบไม่เปลี่ยน แปลว่ารถจอดนิ่ง คือรถติดจริง
• ถ้ารถหนาแน่นแต่ภาพเปลี่ยนมาก แปลว่ารถเยอะแต่ยังเคลื่อนตัว
วิธีนี้ช่วยแก้จุดอ่อนที่ว่า "พื้นที่ถูกรถเต็ม" ไม่ได้แปลว่ารถติดเสมอไป

ข้อจำกัดที่ควรเขียนในรายงาน:
• ตอนกลางคืนไฟหน้ารถทำให้ผลเพี้ยน (ใช้ค่าความสว่างเฉลี่ยตรวจว่าเป็นกลางคืน แล้วติดธงไว้)
• ฝนตกหรือถนนเปียกมีเงาสะท้อน
• เงาต้นไม้และเงาอาคาร
• กล้องอาจถูกหมุนหรือซูม (ตรวจด้วย template matching ได้)

ตัวอย่างโค้ด Collector แบบพื้นฐาน

```
import os, time, datetime, requests

URL = "https://ipcamlive.com/player/snapshot.php?alias=6695edb811079"
INTERVAL = 300  # วินาที (5 นาที)

while True:
    now = datetime.datetime.now()
    folder = f"data/raw/{now:%Y-%m-%d}"
    os.makedirs(folder, exist_ok=True)
    try:
        r = requests.get(URL, timeout=20)
        if r.status_code == 200 and r.headers.get("Content-Type", "").startswith("image"):
            with open(f"{folder}/{now:%H%M%S}.jpg", "wb") as f:
                f.write(r.content)
    except requests.RequestException as e:
        print(now, "error:", e)
    time.sleep(INTERVAL)
```

3. เนื้อหาในสไลด์ที่เชื่อมกับโปรเจกต์

สัปดาห์	เรื่องในสไลด์	ใช้ในโปรเจกต์ตรงไหน
W1	Digital image, Gray/Binary image, Brightness, Contrast	แปลงภาพเป็น grayscale; ใช้ค่า brightness ตรวจกลางวัน/กลางคืน; contrast บอกคุณภาพภาพ (เช่น ฝนหรือหมอก)
W2	Histogram, Histogram Equalization, Contrast stretching, **Thresholding**	ปรับภาพให้สม่ำเสมอทุกช่วงเวลา; ใช้ threshold (และ Otsu) แยกรถออกจากถนน
W3	Noise, **Median / Average filter**	ลด noise จากการบีบอัด JPEG; สร้างภาพถนนว่างด้วย temporal median
W4	**Edge detection** (Sobel, Prewitt, Laplacian)	ความหนาแน่นของขอบเป็นตัวชี้วัดเสริม: ถนนเปล่าเรียบ ขอบน้อย ส่วนรถมีขอบเยอะ
W5	**Color model** RGB / HSV / HSI / YCbCr	ถนนเป็นสีเทาซึ่งมีค่า Saturation ต่ำ รถสีจัดจึงแยกออกง่าย; ใช้ช่อง Y/V ช่วยรับมือเงา
W6	**Morphology** (Erosion, Dilation, Opening, Closing)	ทำความสะอาด mask ของรถ และสร้าง/ปรับ ROI
W7	**Geometric transform** (Translation, Scaling, Rotation, Shearing)	ปรับ perspective ของเลน และจัดแนวภาพเมื่อกล้องขยับ
W8	**Template matching**	จับคู่จุดสังเกตที่ไม่ขยับ (เช่น โลโก้หรือซุ้มประตู) ตรวจว่ากล้องหมุนหรือซูมไหม แล้วเลื่อน ROI ตามอัตโนมัติ
W9	**Segmentation, K-means**	ขั้นหลักของงาน: แยกรถกับถนน และเทียบวิธี threshold กับ K-means (K=2 หรือ 3)

โปรเจกต์นี้ใช้เนื้อหาครบทั้ง 9 สัปดาห์ จึงเป็นจุดขายตอนนำเสนอได้

4. การแบ่งงาน 5 คน (ทุกคนได้ทำ image processing)

> ก่อนแยกกันทำ ให้ตกลง **interface กลาง** ร่วมกันก่อน ทุกคนจะได้ทำงานไปพร้อมกันได้ เช่น
> `preprocess(img) → img · segment(img, bg, mask) → binary · occupancy(binary, mask) → float`

คน	ส่วนที่รับผิดชอบ	เนื้อหา IP
1. ข้อมูลและเรขาคณิต	สคริปต์ดึงภาพ, วาด ROI แต่ละเลน, perspective warp, template matching ตรวจกล้องขยับ	W1, W7, W8
2. Preprocessing	ตัดส่วนเวลาที่ประทับ, grayscale, median/average filter, histogram equalization, ตรวจกลางวัน/กลางคืนจาก brightness	W1, W2, W3
3. Segmentation หลัก	สร้างภาพถนนว่าง (temporal median), absdiff, Otsu threshold, ทดลอง K-means เทียบกัน	W2, W9
4. Feature และ Morphology	edge density (Sobel), ช่อง Saturation ใน HSV, opening/closing ทำความสะอาด mask, วัดการเคลื่อนไหวจาก 2 ภาพ	W4, W5, W6
5. วัดผลและสรุป	คำนวณ occupancy, ปรับเกณฑ์ระดับ, ประเมิน accuracy/confusion matrix, กราฟ 24 ชม., heatmap, ภาพ overlay สำหรับนำเสนอ	W1, W2

งานที่ทำร่วมกันทุกคน: ติดป้ายระดับรถติดให้ภาพประมาณ 100 ภาพเพื่อใช้เป็นคำตอบจริง (ground truth) และเขียนส่วนของตัวเองในรายงาน

ลำดับเวลาที่แนะนำ

ช่วงเวลา	งาน
สัปดาห์ 1	คนที่ 1 เปิดตัวดึงภาพทันที คนอื่นใช้ภาพชุดแรกทดลองอัลกอริทึมของตัวเองใน notebook
สัปดาห์ 2	ต่อทุกส่วนเข้าเป็น pipeline เดียว
สัปดาห์ 3	ใช้ข้อมูล 1–3 วันเต็มปรับเกณฑ์และประเมินผล
สัปดาห์ 4	ทำกราฟ รายงาน และสไลด์นำเสนอ

โครงสร้างโฟลเดอร์ที่แนะนำ

```
MiniProject/
├── collector.py        # ① ดึงภาพทุก 5 นาที
├── roi_tool.py         # ③ วาด ROI แต่ละเลน → masks/*.png
├── preprocess.py       # ②
├── segment.py          # ④ background + threshold / K-means
├── features.py         # ⑤ morphology, edge, motion
├── analyze.py          # ⑥–⑧ occupancy → CSV → กราฟ
├── data/raw/YYYY-MM-DD/HHMMSS.jpg
├── masks/
└── results/occupancy.csv, charts/
```